SabPaisa Automation Test Suite — Handover Guide

Intended Audience: Manual testers or team members new to automation.
Purpose: Enable anyone to run, maintain, and understand this project without prior automation knowledge.

Table of Contents

1. What This Project Does
2. What is Test Automation?
3. Glossary of Terms
4. Setting Up Your Computer (First Time)
5. Opening the Project in PyCharm
6. Where to Change Things
7. How to Run Tests — Step by Step
8. Understanding Results
9. Running in CI/CD
10. What Each Test Does
11. Troubleshooting
12. Resources

---

1. What This Project Does

This project **automatically tests the SabPaisa Payment Gateway checkout page**. Instead of a human clicking through every payment option manually, the automation:

- Opens a browser (Chrome)
- Goes to the SabPaisa checkout URL for a merchant (like CHIN36)
- Fills the customer form (name, email, phone, amount)
- Clicks each payment mode (UPI, Cards, Netbanking, Wallets, Offline)
- Tries every bank, every wallet, and every combination
- Records what worked and what failed
- Produces an **Excel report** and an **Allure HTML report** showing pass/fail per item

Why? Running 68 tests manually takes a full day. The automation does it in 15 minutes and catches regressions before customers see them.

2. What is Test Automation?

Think of it like a robot that performs manual testing on your behalf. You tell it:
- "Click this button"
- "Fill this field"
- "Verify this message appears"

...and it does those steps over and over, much faster than a human.

Manual testing: You open checkout, try paying with ICICI Bank, see what happens.
Automated testing: A script opens checkout, clicks ICICI Bank, records pass/fail, then moves to the next bank automatically — repeating for 30 banks.

The automation for this project is written in **Python** (a programming language) using **Playwright** (a browser automation tool).

3. Glossary of Terms

Term	Meaning (plain English)
Python	The programming language this project is written in. Like Java or JavaScript.
pip	The tool that installs Python packages (`pip install ...`).
Playwright	Tool that controls the browser — clicks buttons, fills forms, reads text.
Pytest	The framework that runs tests. You type `pytest` and it runs everything.
Test	A single scenario (e.g., "CHIN36 Cards payment should reach bank page").
Test suite	A collection of tests (68 in this project).
POM (Page Object Model)	A design pattern — each page has a file describing what's on it. See `pages/checkout_page.py` .
Locator	How automation finds an element on the page (e.g., `img[alt='PhonePe']` for the PhonePe icon).
Assertion	A check that must pass (e.g., "Convenience Charges should be visible").
Allure	Reporting tool — shows test results with screenshots and step-by-step logs.
Headless	Browser runs invisibly (faster). Opposite: **Headed** = you see the browser.
CI/CD	Automatic test runs on every code push. Examples: Jenkins, GitHub Actions, Bitbucket.
Fixture	Shared test setup (e.g., "open a browser before each test").
Merchant / Client code	Identifier like CHIN36 = Chinmay Bharat Academy. Each has different payment options enabled.
Fee Forward	A merchant config flag. YES = show convenience charges on checkout; NO = hide them.
Gateway	The bank's payment page (e.g., MobiKwik's page after you click Pay).
Challan	Offline payment receipt / confirmation page.

4. Setting Up Your Computer (First Time)

Do these once. After that, you only run tests.

Step 1 — Install Python 3.11

1. Download from https://www.python.org/downloads/release/python-3119/
2. Run installer, **check "Add Python to PATH"** at the bottom, then click Install
3. Open Command Prompt (cmd) or PowerShell, type `python --version`
4. Should show `Python 3.11.x`

Step 2 — Install Git (if not already)

1. Download from https://git-scm.com/downloads
2. Run installer with default options
3. Verify: open cmd, type `git --version`

Step 3 — Get the Project

```
git clone <your-git-repo-url> playwright_checkout_project
cd playwright_checkout_project
```

Or if you received a zip: extract it, open cmd in that folder.

Step 4 — Install Project Dependencies

From inside the project folder:

```
pip install -r requirements.txt
playwright install chromium
```

This downloads:
- All Python packages (pytest, Playwright, Allure plugin, openpyxl for Excel)
- The Chromium browser Playwright uses

Step 5 — Install Allure Command-Line (for reports)

Windows (recommended — using Scoop):

```
scoop install allure
```

If you don't have Scoop: install from https://scoop.sh/

Alternative (using npm):

```
npm install -g allure-commandline
```

Verify: `allure --version`

Step 6 — Install PyCharm (optional but recommended)

PyCharm is the code editor for Python. Download from https://www.jetbrains.com/pycharm/download/ (Community Edition is free).

You are now ready to run tests.

---

5. Opening the Project in PyCharm

1. Launch PyCharm
2. File → Open → pick the `playwright_checkout_project` folder → OK
3. PyCharm may ask to "Configure interpreter" — click it, select Python 3.11
4. Wait for PyCharm to index (green progress bar at bottom right)
5. In the left sidebar (Project pane) you'll see the folders: `config/`, `data/`, `pages/`, `tests/`, `utils/`

Key files to know

File	What it contains
`data/checkout_data.json`	Default merchant ID, customer details, card, UPI
`data/clients.json`	List of merchants for batch runs
`config/settings.json`	Environment (Staging/Production), headless, slow-mo, timeout
`tests/test_regression_suite.py`	All 68 tests
`pages/checkout_page.py`	The biggest file — all checkout page logic
`run_parallel_clients.py`	Batch runner that produces Excel + Allure HTML
`README.md`	Full technical reference
`HANDOVER_GUIDE.md`	This document

---

6. Where to Change Things

Most common edits a manual tester would make:

Change test amount (e.g., ₹45 → ₹100)

1. Open `data/checkout_data.json` in PyCharm
2. Find the line: `"amount": "45",`
3. Change `"45"` to `"100"`
4. Save (Ctrl+S)
5. Rules:
   - Keep the quotes: `"100"` not `100`
   - No symbols: `"100"` not `"₹100"`
   - Decimals OK: `"99.99"`

Change default merchant

In the same file, change:

```
"merchant_id": "CHIN36",
```

to another merchant code (e.g., `"SUBI79"`).

Change environment (Staging ↔ Production)

1. Open `config/settings.json`
2. Change:
   `json "environment": "Staging",`
   to `"Production"` when needed.

Add a new merchant to batch runs

1. Open `data/clients.json`
2. Add a line:
   `json {"merchant_id": "NEWCODE", "environment": "Staging"},`

Change browser speed

In `config/settings.json`:
- `"slow_mo": 500` → 500 ms delay between actions. Set to `0` for fastest, `1000` for slower.
- `"headless": false` → browser visible. Set to `true` to hide it (faster).

---

7. How to Run Tests — Step by Step

Open Command Prompt / PowerShell in the project folder. Copy-paste the command you want.

7.1 Run ONE specific test (watch in browser)

```
pytest tests/test_regression_suite.py::TestR11FeeForward::test_chin36_charges_visible --
```

What this does:
- Opens Chrome
- Tests CHIN36 Fee Forward — confirms Convenience Charges are visible
- Browser closes when done; you'll see PASSED or FAILED at the bottom

7.2 Run all tests for ONE mode

```
# Netbanking section only
pytest tests/test_regression_suite.py::TestR5Netbanking --headed --slowmo 500

# Wallets section only
pytest tests/test_regression_suite.py::TestR6Wallets --headed --slowmo 500

# Offline section only
pytest tests/test_regression_suite.py::TestR7Offline --headed --slowmo 500
```

7.3 Run all CHIN36 tests (all modes for CHIN36)

```
pytest tests/test_regression_suite.py -k "chin36" --headed --slowmo 500
```

This runs 8 tests: UPI, Cards ×2, Netbanking ×2, Wallet, Offline ×3 — all for CHIN36.

7.4 Run the dynamic per-bank/per-wallet flows

These test **every** bank/wallet the client has, fresh session per item:

```
# All three dynamic flows at once
pytest tests/test_regression_suite.py -k "per_bank_flow or per_flow" --headed --slowmo 5
```

7.4.1 Test ONE specific bank (e.g. PNB, ICICI, HDFC)

Use the `BANK_FILTER` environment variable in PowerShell:

```
# Test only PNB end-to-end (~2 min)
$env:BANK_FILTER="PNB"
pytest tests/test_regression_suite.py::TestR5Netbanking::test_netbanking_complete_flow -
```

Other examples:
- `$env:BANK_FILTER="ICICI"` — runs only ICICI Bank
- `$env:BANK_FILTER="HDFC"` — runs only HDFC Bank
- `$env:BANK_FILTER="State Bank"` — runs SBI
- `$env:BANK_FILTER="Equitas"` — runs Equitas Bank
- `Remove-Item Env:BANK_FILTER` — reset (run all banks again)

Easy way — save as `run_one_bank.ps1` in project root:

```
$env:BANK_FILTER = "PNB"
pytest tests/test_regression_suite.py::TestR5Netbanking::test_netbanking_complete_flow -
```

Then run with `.\run_one_bank.ps1`. To test a different bank, edit the file's bank name and re-run.

7.5 Full regression (all 69 tests)

```
# With visible browser (slow, for review)
pytest tests/test_regression_suite.py --headed --slowmo 500 --alluredir=reports/allure-r

# Headless (fastest)
pytest tests/test_regression_suite.py --alluredir=reports/allure-results
```

7.6 Client-ready Excel + Allure HTML (RECOMMENDED for sharing)

```
# One client
python run_parallel_clients.py --clients CHIN36 --workers 1

# Two clients in parallel (both run simultaneously)
python run_parallel_clients.py --clients CHIN36,SUBI79 --workers 2

# Smoke only (faster — E2E tests only)
python run_parallel_clients.py --clients CHIN36,SUBI79 --workers 2 --smoke
```

This produces:
- `reports/Client_Regression_Report_<timestamp>.xlsx` — **5-sheet Excel** with per-mode PASS/FAIL

- `reports/allure-report-CHIN36/` — clickable HTML per client
- `screenshots/` — PNGs of every step

---

8. Understanding Results

8.1 Console output

After running, pytest prints:

```
======================== 68 passed in 1243.45s ========================
```

or

```
==================== 65 passed, 3 failed in 1243.45s ====================
```

Green `passed` = works. Red `failed` = broken — check Allure for details.

8.2 Allure HTML report

```
allure serve reports/allure-results
```

This opens your browser with a live report showing:
- **Overview** — total pass/fail counts
- **Suites** — tests grouped by R1–R13 sections
- **Graphs** — trends, test duration
- **Click any test** to see every step, screenshots, error messages

8.3 Excel report

Open `reports/Client_Regression_Report_<timestamp>.xlsx`:

Sheet	What it shows
Summary	Total pass/fail per client, overall mode status
Cards Detail	Debit Card + Credit Card per client
Netbanking Detail	30 banks × client (green = PASS, red = FAIL, grey = N/A)
Wallets Detail	7 wallets × client
Offline Detail	6 offline combos (Cash×4 + RTGS + IMPS) × client

Color key:
- 🟩 Green = PASS (bank gateway opened / flow worked)
- 🟥 Red = FAIL (error or gateway didn't open)
- ⬜ Grey = N/A (client doesn't offer this bank/wallet)

Example interpretation:
If CHIN36's Wallets Detail shows 🟩 MobiKwik, 🟩 Airtel, 🟥 PhonePe, 🟥 AmazonPay, ⬜ FreeCharge, ⬜ Jio, ⬜ OLA — this means:
- CHIN36 supports 4 wallets (not 7)
- MobiKwik + Airtel work
- PhonePe + AmazonPay currently failing (need investigation)
- FreeCharge/Jio/OLA aren't enabled for CHIN36

---

9. Running in CI/CD

CI/CD = Continuous Integration / Continuous Delivery. It runs tests automatically on every code push. Three options are pre-configured.

9.1 Jenkins (on-premise)

File: `Jenkinsfile` in project root.

How to set up:
1. On Jenkins dashboard → **New Item** → enter name (e.g., `sabpaisa-regression`) → select **Pipeline** → OK
2. In the job config:
- **Pipeline** section → select "Pipeline script from SCM"
- SCM: Git
- Repository URL: your Git URL
- Script Path: `Jenkinsfile`
3. Click **Save**
4. Click **Build Now** on the job page
5. When finished, click the **Allure Report** link on the build page to view

To modify what tests run in Jenkins: edit `Jenkinsfile`:

```
bat 'pytest tests/test_regression_suite.py -k "chin36" --alluredir=allure-results'
```

Plugins needed on Jenkins: "Allure Jenkins Plugin" (install via Manage Jenkins → Plugins)

9.2 GitHub Actions

File: `.github/workflows/regression.yml`

How to trigger manually:
1. Go to your GitHub repo → **Actions** tab
2. Click "Regression Suite" workflow in left sidebar
3. Click **Run workflow** → pick branch → Run workflow

Auto-triggers: push to `main` / `develop`, pull requests, scheduled runs (Mon-Fri 6 AM UTC).

How to see results:
- Actions tab → click the run → scroll to **Artifacts** → download `allure-results`
- Extract on your machine, then: `allure serve allure-results`

9.3 Bitbucket Pipelines

File: `bitbucket-pipelines.yml`

Auto-triggers: every push + two manual pipelines: `smoke-test` (fast) and `full-regression` (all 68 tests).

How to trigger manually:
1. Go to your Bitbucket repo → **Pipelines** → **Run pipeline**
2. Choose `smoke-test` or `full-regression` → **Run**

How to see results: Pipelines page → click the pipeline → **Artifacts** → download allure-results zip.

9.4 Docker (local container)

Build once:

```
docker build -t sabpaisa-tests .
```

Run anytime:

```
docker run --rm -v $(pwd)/reports:/app/reports sabpaisa-tests
```

The `-v` flag mounts the `reports/` folder so test outputs land on your local disk. Then view with `allure serve reports/allure-results`.

Run with custom args:

```
docker run --rm -v $(pwd)/reports:/app/reports sabpaisa-tests \
  pytest tests/test_regression_suite.py -k "chin36" --alluredir=reports/allure-results
```

---

10. What Each Test Does

R1 — Merchant Config (6 tests)

Checks the "Configure" page — where you enter merchant ID and click Fetch to load config. Covers invalid merchant, special characters, and skipping Fetch.

R2 — Customer Form (8 tests)

Checks the form where customer enters name, email, phone, amount. Covers:
- **R2.1** Happy path — fill + proceed
- **R2.2** Empty fields — should show validation error
- **R2.3** Invalid email — should show error
- **R2.4** Zero amount — should reject
- **R2.5** Negative amount (-100) — should reject
- **R2.6** Large amount (23000) — should accept and show on checkout
- **R2.7** Decimal amount (99.99) — should accept
- **R2.8** Hindi input (राम कुमार) — should accept

R3 — UPI (2 tests)

Tests UPI mode: select UPI tab, click Generate QR, verify QR code displays.

R4 — Cards (5 tests)

Card form: fill number/name/expiry/CVV, submit. Covers invalid card, empty card, expired card.

R5 — Netbanking (10 tests)

The largest mode. Tests:
- Search for a bank
- Show all banks / hide all banks
- Click each bank in the popular grid
- **R5.11** Dynamic per-bank flow: fresh session per bank → click → Pay → verify gateway opens

R6 — Wallets (3 tests)

- R6.1 Wallet grid appears
- R6.2 Click every wallet sequentially
- **R6.3** Dynamic per-wallet E2E: fresh session per wallet → click → Pay → verify gateway opens

R7 — Offline (6 tests)

Cash / RTGS / IMPS sub-tabs. Tests:
- R7.1–4 Basic sub-tab selections
- **R7.5** Per-bank walkthrough with screenshots + challan verification
- **R7.6** Dynamic per-bank across Cash/RTGS/IMPS

R8 — Mode Switching (4 tests)

Rapid switching between modes, card data persistence, page reload behavior.

R9 — E2E Smoke (5 tests)

Quick end-to-end tests for each mode — one successful flow per mode.

R10 — Full E2E per Client (16 tests)

CHIN36 (8 tests) + SUBI79 (8 tests) — complete mode-by-mode run per client.

R11 — Fee Forward (3 tests) ⭐ NEW

- **R11.1** CHIN36 (Fee Forward = YES) → Convenience Charges MUST be visible on checkout
- **R11.2** SUBI79 (Fee Forward = NO) → Convenience Charges MUST be hidden
- **R11.3** Math: Total Amount = Order Amount + Convenience Charges

R12 — Fetch Validation (3 tests) ⭐ NEW

- **R12.1** Valid merchant → Fetch populates API URL + green indicator visible
- **R12.2** Invalid merchant → API URL stays empty, error shown
- **R12.3** User skips Fetch → API URL empty (can't reach valid checkout)

R13 — Language Dropdown (3 tests) ⭐ NEW

- **R13.1** Language dropdown opens with options
- **R13.2** Switch to Hindi → button label changes
- **R13.3** Switch through all available languages

---

11. Troubleshooting

"Python not recognized"

Python isn't on PATH. Reinstall Python and check "Add to PATH" option.

"playwright install" fails

Try: `python -m playwright install chromium`

Tests hang / browser doesn't open

The browser was in a bad state from a previous run. Close all Chromium windows, try again.

"allure: command not found"

Install Allure CLI (see Step 5 in Section 4).

All tests fail with "Target page closed"

Your `page` fixture is timing out. Try running a smaller batch: `pytest tests/... -k "R3"` first.

Excel file locked when running batch runner

You have an old Excel report open in Microsoft Excel. Close it first.

"Module not found" errors

Dependencies not installed. Run: `pip install -r requirements.txt`

Clean everything and retry

```
rm -rf __pycache__ */__pycache__ .pytest_cache
rm -rf reports/allure-results* reports/allure-report-*
rm -f screenshots/*.png
```

---

12. Resources

Documentation

- **Playwright docs** — https://playwright.dev/python/
- **Pytest docs** — https://docs.pytest.org/
- **Allure docs** — https://allurereport.org/
- **Python tutorial** (official) — https://docs.python.org/3/tutorial/

Project-specific

- **README.md** — technical reference with all commands
- **HANDOVER_GUIDE.md** — this document (beginner-friendly)
- **Checkout_Test_Cases.xlsx** — original manual test case reference (in project root)

Learning Python basics (if needed)

- Free video course: https://www.youtube.com/watch?v=_uQrJ0TkZlc ("Python for Everybody")
- Interactive practice: https://www.learnpython.org/

Common first tasks for a new tester

1. Run the first time: `pytest tests/test_regression_suite.py -k "chin36" --headed --slowmo 500`
2. Open the Allure report: `allure serve reports/allure-results`
3. Change the amount in `data/checkout_data.json` → re-run
4. Run the batch runner: `python run_parallel_clients.py --clients CHIN36 --workers 1`
5. Open the Excel and see the 5 sheets

Final Checklist (for the new tester)

- [ ] Python 3.11 installed and on PATH
- [ ] Project cloned / extracted
- [ ] `pip install -r requirements.txt` ran successfully
- [ ] `playwright install chromium` ran successfully
- [ ] Allure CLI installed (`allure --version` works)
- [ ] Opened project in PyCharm
- [ ] Ran one test successfully (e.g. `pytest -k "R3" --headed`)
- [ ] Viewed the Allure report
- [ ] Ran the batch runner and viewed the Excel
- [ ] Changed the amount in `checkout_data.json` and re-ran

You are ready. Everything else you need is in the README or this guide.

This document was generated as part of the project handover. Keep it with the codebase. Update it when new modes or scenarios are added.